Fire Assignment 12

by Lorraine Gaudio

Lesson generated on November 15, 2025

BOISE STATE UNIVERSITY

Contents

1. ☒ Point-Boost Activity

 Idaho is well known for its wildfires. They play a ecological role, shaping our forests and grasslands while presenting significant hazards to communities and resources. In recent years, the Idaho landscape has experienced frequent and sometimes severe wildfire seasons due to both natural and human activity. While fires are part of the natural ecosystem, they have become more destructive as the climate warms. Wildfire preparedness and management is a continuing challenge for the state. For more information this research at Boise State.

How far could a new wildfire run in 8 hours? How many new wild fire ignitions will grow larger than 10 km (straight line) in 8 hours?

This is an optional two-part point-boost activity. You can work in groups but each person submits their own work independently. Please record group members within your document.

1.1 Assignment Directions

Read the background material below, then complete the tasks that follow. Replace each ____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment.

1. **Technical Skill Points**

You could earn *up to* **10** points in Technical skill. This assignment will require code covered in modules 1 through 13. You will load a CSV file, manipulate the data frame with dplyr verbs, simulate random numbers and visualize results. Additionally, you will be asked to use functions that were not covered in any modules. You will need to look up the function documentation and figure out how to use them on your own. The submission must include an HTML report and RData of the environment.

2. **Learning Skill Points**

You could earn up to **10** points Learning skill on this optional Point Boost assignment!

- Use the template Goal → Code → Checks → Comment
- □ Cite resources (including referencing lessons, peers, instructor guidance, internet websites, AI summaries, the help file in R) in memos. State specifically what or who helped and in what ways. Always compose your own code (no copy and paste).
- □ Reflect on learning process, connects ideas, discusses challenges, and describes improvement strategies.

- □ Format your document with headings, code chunks, and narrative text. Use proper grammar, spelling, and punctuation. Include a document title, author name, and □ date in the heading of your report.
- □ Code trials show how you explored ideas, asked questions independently of the assignment. Include code comments to explain your thinking.
- □ Growth mindset means you demonstrate effort, persistence, and a positive attitude toward learning new skills. You show courage.

3. **Signature Assignment Option**

You may choose to direct these points to your Signature Assignment requirement. If you do, you **must** complete all tasks and submit a polished HTML document with clear code, thorough comments, and well-written explanations.

When finished, □ Knit or Render your report to HTML, and upload with your RData to Canvas by or before December 12th.

1.2 Dataset

Wildfire managers need tools that show where fires are more likely and how intense they could become. The U.S. Forest Service's Wildfire Hazard Potential (WHP) is one such tool: a nationwide raster layer where each grid cell stores a unitless index of relative wildfire hazard (higher value = higher hazard). For this activity we use an Idaho-only WHP file prepared by Dr. Matt Williamson SPASES Lab Page.

Figure 1 shows the Idaho WHP raster created in R with the packages terra (raster operations), sf (vector data), tidyverse (data wrangling), tidyterra (ggplot helpers for rasters), and tigris (state/county boundaries). The GeoTIFF uses an Albers Equal Area projection (units: meters) with 240 × 240 m cells. Within Idaho, valid cells cover about 216,315 km²; cells outside the state within the file's bounding box are NoData. WHP values in this subset range from 0 to 64,185.656; the median is about 397, and the upper tail is steep (e.g., 95th percentile ≈ 7,753, 99th ≈ 22,429).

Formal dataset citation

Dillon, Gregory K.; Gilbertson-Day, Julie W. 2020. Wildfire Hazard Potential for the United States (270-m), version 2020. 3rd Edition. Forest Service Research Data Archive. https://doi.org/10.2737/RDS-2015-0047-3.

Idaho subset (whp_id.tif) prepared and provided by Matt Williamson, SPASES Lab, Boise State University.

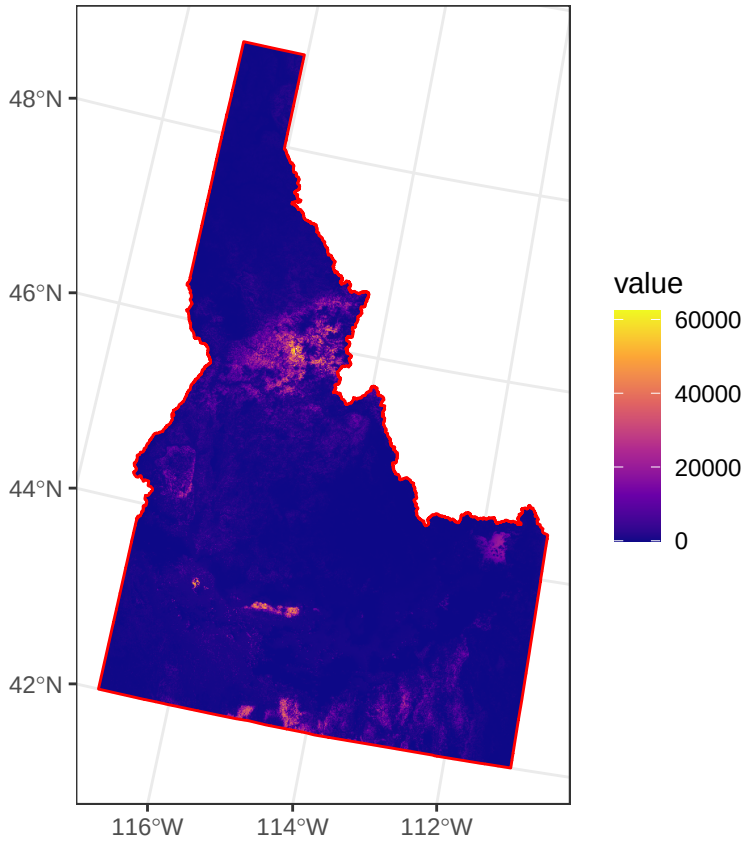

Figure 1.1: Figure 1. Idaho Wildfire Hazard Potential (WHP), continuous index (unitless) from the USFS WHP v2020 raster, clipped to the Idaho boundary. Colors use a viridis gradient (dark purple = lower WHP; yellow = higher WHP); values span approximately 0–64,186. Raster resolution is 240 m in an Albers Equal Area projection; NoData cells are transparent. Red outline shows the TIGER/Line state boundary reprojected to the raster CRS. Idaho subset prepared by Matt Williamson (SPASES Lab, Boise State University); original dataset: Dillon & Gilbertson-Day (2020), USFS Research Data Archive (DOI: 10.2737/RDS-2015-0047-3).

2. Part A

Explore and wrangling of the raw WHP sample data.

2.1 Task 1

 Goal: Read the whp_raw_sample.csv using readr, storing it in your environment as county_raw.

 Code

```
library(readr)
county_raw <- readr::read_csv("whp_raw_sample.csv")
```

```
#  Verify Ideas
dplyr::glimpse(county_raw)
colnames(county_raw)
summary(county_raw$whp)
nrow(county_raw)
```

 Comment: county_raw is a 150,000 row tibble of raw WHP cell values with county and per-cell area.

whp – The Wildfire Hazard Potential value for a single sampled raster cell. This is a unitless, continuous index (higher = greater relative hazard). Values come directly from the WHP GeoTIFF.

county – The county name (e.g., "Idaho", "Twin Falls") obtained by spatially joining each sampled cell to the TIGER/Line county boundary. Use for readable reports.

county_fips – The county's 3-digit FIPS code from TIGER/Line. *Option: To create the standard 5-digit FIPS for Idaho, prepend the state code 16: e.g., 049 → 16049. Need to use the 5-digit code when joining to external datasets (ACS, MTBS summaries, etc.).*

cell_area_km2 – The area (in square kilometers) represented by that raster cell, computed from the raster's equal-area projection. With 240 m × 240 m cells this is ~0.0576 km²; small numeric differences reflect rounding. Summing this column by any grouping yields true area totals, and dividing by the statewide sum yields area shares.

 Task 2 through 5 Goal: Compute quantile cut-points, label categories, and estimate probability (statewide area share) from this.

2.2 Task 2

 Goal: Use dplyr::summarize() to report the following about county_raw:

- Number of rows
- Number of missing values in whp
- Minimum value of cell_area_km2
- Maximum value of cell_area_km2

```
# Code: for dplyr::summarize
county_raw %>%
  summarize(
    n_rows   = n(),
    n_na_whp = sum(is.na(whp)),
    min_area = min(cell_area_km2),
    max_area = max(cell_area_km2)
  )
```

 Comment:

2.3 Task 3

Compute WHP quantile edges (category cutpoints)

 Goal: Use quantile() function from base R's stats package to compute the quantiles of the whp column in county_raw. Store the result as e for "edges".

Create the seven category edges using the quantiles (0, .25, .50, .75, .90, .95, .99, 1.00).

 The SYNTAX: quantile(x, probs, na.rm)

```
# Code: Base R stats::quantile
e <- stats::quantile(
  county_raw$whp,
  probs = c(0, .25, .50, .75, .90, .95, .99, 1),
  na.rm = TRUE
)
```

```
# Verify
e
```

 Comment: The e object now contains the cut-points for categorizing WHP into quantile-based bins.

- e[1] = minimum WHP (0th percentile)
- e[2] = 25th percentile

- e[3] = 50th percentile (median)
- e[4] = 75th percentile
- e[5] = 90th percentile
- e[6] = 95th percentile
- e[7] = 99th percentile
- e[8] = maximum WHP (100th percentile)

2.4 Task 4

Add a Category label

 Goal: Use dplyr::mutate() and dplyr::case_when() to turn the numeric whp values in county_raw into an ordered categorical variable called Category, using the quantile edges e you computed in Task 3. Store the resultant data set as county_cats.

1. Start from county_raw.
2. Use mutate() to add a new variable Category.
3. Use case_when() to assign a label based on which interval of e each whp value falls into.
4. Convert Category to an ordered factor so the levels follow the intended risk order.

Each row will be assigned exactly one of these labels based on its whp value:

- If a row's whp value is *at least* the minimum WHP AND *less than* the 25th percentile, label it "Very Low".
- If whp is *at least* the 25th percentile AND *less than* the 50th percentile (median), label it "Low".
- If whp is *at least* the 50th percentile AND *less than* the 75th percentile, label it "Moderate".
- If whp is *at least* the 75th percentile AND *less than* the 90th percentile, label it "High".
- If whp is *at least* the 90th percentile AND *less than* the 95th percentile, label it "Very High".
- If whp is *at least* the 95th percentile AND *less than* the 99th percentile, label it "Extreme".
- If whp is *at least* the 99th percentile AND *less than or equal to* the maximum WHP, label it "Ultra (Top 1%)".

If a value of whp does not fall in any of these ranges (e.g., NA), Category should be NA.

Hint: All intervals are closed on the left (>=) and open on the right (<), except the last one, which is closed on both sides (>= and <=).

```
# □ Code: dplyr::mutate + dplyr::case_when
county_cats <- county_raw %>%
 mutate(
  Category = case_when(
   whp >= e[1] & whp <  e[2] ~ "Very Low",
   whp >= e[2] & whp <  e[3] ~ "Low",
   whp >= e[3] & whp <  e[4] ~ "Moderate",
   whp >= e[4] & whp <  e[5] ~ "High",
   whp >= e[5] & whp <  e[6] ~ "Very High",
   whp >= e[6] & whp <  e[7] ~ "Extreme",
   whp >= e[7] & whp <= e[8] ~ "Ultra (Top 1%)",
   TRUE                      ~ NA_character_
  ),
  Category = factor(
   Category,
   levels = c("Very Low","Low","Moderate","High","Very High","Extreme","Ultra (Top 1%)"),
   ordered = TRUE
  )
 )
```

```
# □ Verify
county_cats %>% count(Category)
```

□ Comment: I used case_when to assign labels by WHP range. Because the CRS is equal-area and cell size is uniform, counts approximate area. The cell_area_km2 column lets you compute true area easily.

2.5 Task 5

□ Goal: Use dplyr::group_by() and dplyr::summarize() to aggregate total area by Category, then compute Probability as each category's share of the total statewide area. Store the result as prob_tbl.

1. Start from county_cats (the data set you created in Task 4, which already has Category and cell_area_km2).
2. Use group_by() so that all rows in the same Category are treated as a single group.
3. Use summarize() to compute, for each Category:
   - area_km2: the total area in square kilometers, as the sum of cell_area_km2 across all cells in that category.
   - n_cells: the number of grid cells in that category using n().
   - Use .groups = "drop" so that the result is no longer grouped after summarizing.
4. Use mutate() to create a new variable called Probability that is the area_km2 divided by sum(area_km2).

This makes Probability the fraction of statewide area that falls in each Category. The probabilities across all categories should sum to 1 (up to rounding error).

5. Use select() to keep only the columns, in this order: Category, Probability, area_km2, n_cells.

```
# □ Code: dplyr::group_by + dplyr::summarize + dplyr::mutate
prob_tbl <- county_cats %>%
  group_by(Category) %>%
  summarize(
    area_km2 = sum(cell_area_km2),
    n_cells  = n(),
    .groups = "drop"
  ) %>%
  mutate(
    Probability = area_km2 / sum(area_km2)
  ) %>%
  select(Category, Probability, area_km2, n_cells)
```

```
# □ Verify
prob_tbl
sum(prob_tbl$Probability)
```

□ Comment: The prob_tbl tibble now summarizes total area and cell counts by WHP category, along with the statewide area share (Probability) for each category.

3. Part B

The table below summarizes the quantile-based wildfire hazard categories derived from Idaho's continuous USFS WHP raster and the distance distributions used for an 8-hour run. The **Category** column lists seven bins (Very Low to Ultra) defined from the raster's cumulative distribution; the WHP index itself is unitless. The **Probability** column is the statewide area share for each bin—expressed as a percent of valid raster area (NA excluded)—and is interpreted as the chance that a random ignition falls in that category.

The table also defines a simple fire-spread scenario using "plausible values" derived from ChatGPT for an 8-hour straight-line distance by hazard category. These distances are not estimated from the WHP raster. They are modeling assumptions chosen for this assignment. Specifically, we assume that (1) ignitions are equally likely anywhere within the valid WHP raster (so Probability reflects area share), (2) higher WHP categories correspond to larger 8-hour straight-line spread distances, and (3) within each category, 8-hour distances follow either a zero-truncated normal distribution with the given mean and SD or a lognormal distribution.

The **Normal mean (km)** and **Normal SD (km)** columns give the mean and standard deviation, in kilometers, for a zero-truncated normal distance model representing straight-line spread over 8 hours. The **Mean log** and **SD log** columns give the parameters μ and σ of a lognormal distance model in which

$$\log(\text{distance in km}) \sim \mathcal{N}(\mu, \sigma^2).$$

Labels "Extreme" and "Ultra" denote upper-tail bins (approximately the 95–99% and 99–100% quantiles, respectively).

Category	Probability	Mean (km)	SD (km)	Mean log	SD log
Very Low	25	2.0	1.0	0.582	0.472
Low	25	3.0	1.0	1.046	0.325
Moderate	25	4.5	1.5	1.451	0.325
High	15	7.0	2.0	1.907	0.280
Very High	5	12.0	3.0	2.455	0.246
Extreme	4	18.0	4.0	2.866	0.220
Ultra (Top 1%)	1	28.0	6.0	3.310	0.212

3.1 Statistical Methods

A continuous random variable is a variable that can take on an infinite number of values within a given range. Unlike discrete random variables, which can only take on specific, separate values (□

like the roll of a die), continuous random variables can take on any value within a certain interval. Examples of continuous random variables include weight or time. In this assignment, we will use distance in km.

Throughout the assignment, you will see mathematical notation. You do not need to understand these equations to complete the tasks. There are there for those who are studying statistics more deeply (me, your instructor).

Let X be the continuous random variable of straight-line kilometer distance in an 8-hour period. Then a probability distribution of X is a function $f(x)$ such that for any two numbers a and b with $a \leq b$,

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx$$

The probability that X takes on a value in the interval $[a, b]$ is given by the area above this interval and under the graph of the density function.

3.2 Log Normal Distribution

In this assignment, we will use the lognormal distribution to model the 8-hour straight-line distance of new ignitions in High wildfire hazard categories. We will explore the list of R functions for the lognormal distribution. You can access the documentation for these functions by running the following command in R:

```
?Lognormal
```

3.2.1 Task 1

plnorm() computes the cumulative distribution function (CDF) for a lognormal distribution. It gives the probability that a lognormally distributed random variable is less than or equal to a certain value.

 Question a: If a new ignition starts in a High-hazard area, what is the probability that its 8-hour straight-line spread is at least 10 km?

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Suppose X has the probability density function $f_{high}(x)$. Then for any two distances a and b with $0 \le a \le b$,

$$P(a \le X \le b) = \int_a^b f_{high}(x)dx.$$

In particular, the probability that the 8-hour straight-line spread is at least 10 km is given by

$$P(X \le 10) = \int_{10}^{\infty} f_{high}(x)dx,$$

and in this problem, $f_{high}(x)$ is the density of a lognormal distribution.

 Goal: Compute the exact probability that a new ignition in a High-hazard area spreads at least 10 km in 8 hours using the plnorm() function. Use the **High** row's **Mean log** and **SD log** in **Table 1**. Set the lower.tail equal to FALSE. Store the result as p_high_exact.

```
# Test: single lognormal, single threshold
p_high_exact <- plnorm(10, meanlog = 1.907, sdlog = 0.280, lower.tail = FALSE)
```

```
# Verify
p_high_exact
p_high_exact * 100  # as a percentage
```

 Comment: The value 0.0788567 indicates a 7.9% chance that a new ignition in a High-hazard area will spread at least 10 km in 8 hours.

3.2.2 Task 2

qlnorm() computes the quantile function (inverse CDF) for a lognormal distribution. It gives the value below which a certain percentage of the distribution falls.

 Question b: We'll treat distances above the 95th percentile as 'extreme fast runs'. For ignitions in High-hazard areas, what 8-hour distance is only exceeded 10% of the time? In other words, what is the 95th percentile run length?

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Suppose X has the probability density function $f_{high}(x)$. The cumulative distribution function (CDF) of X is given by

$$F(x) = P(X \leq x) = \int_0^x f_{high}(t)dt.$$

We define the 95th percentile of X, denoted by $q_{0.95}$ of this distribution to be the value satisfying

$$P(X \leq q_{0.95}) = F(q_{0.95}) = 0.95.$$

equivalently,

$$q_{0.95} = \int_0^{q_{0.95}} f_{high}(x)dx.$$

In particular, the 95th percentile of the 8-hour straight-line spread distance for new ignitions in High-hazard areas is given by

$$q_{0.95} = F^{-1}(0.95),$$

and in this problem $f_{high}(x)$ is the density of a lognormal distribution.

 Goal: Compute the distance such that there is a 95% chance the 8-hour spread is no more than this value and a 5% chance it exceeds it. Use qlnorm(). Set the probability to 0.95 and use the parameters in the **High** row's **Mean log** and **SD log** in **Table 1**.

```
#  Test
q90_high <- qlnorm(0.95, meanlog = 1.907, sdlog = 0.280)
```

```
#  Verify
q90_high
```

 Comment: Based on this model, about 95% of new ignitions in High-hazard areas are expected to spread no more than roughly 10.7 km in 8 hours, and only about 5% would be expected to spread farther than 10.7 km in that time.

3.2.3 Task 3

dlnorm() computes the probability density function (PDF) for a lognormal distribution. It gives the relative likelihood of the random variable taking on a specific value.

 Question c: Under our High-hazard lognormal model, how concentrated the distribution is locally around the shorter 3 km run verses the longer 10 km distance in 8 hours? *Note: Higher density indicates a more likely value in a small neighborhood around that point.*

> Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Suppose X has the probability density function $f_{high}(x)$. Then for any two distances a and b with $0 \leq a \leq b$,
>
> $$P(a \leq X \leq b) = \int_a^b f_{high}(x)dx.$$
>
> The values $f_{high}(x)$ are densities, not probabilities. They represent the height of the distribution at the point x. In particular, the density at 3 km and 10 km are given by
>
> $$f_{high}(3) \text{ and } f_{high}(10).$$

 Goal: Compute the density at 3 km and 10 km using dlnorm(). Use the **High** row's **Mean log** and **SD log** in **Table 1**.

```
# Test

d_3  <- dlnorm(3,  meanlog = 1.907, sdlog = 0.280)
d_10 <- dlnorm(10, meanlog = 1.907, sdlog = 0.280)
```

```
d_3; d_10  # compare heights
```

 Comment: The density at 10 km $f_{high}(10) \approx 0.0525$) is higher than at 3 km ($f_{high}(3) \approx 0.00736$), which implies that values in a small neighborhood around 10 km are more likely than values in a small neighborhood around 3 km under the High-hazard lognormal model.

3.2.4 Task 4

rlnorm() generates random numbers from a lognormal distribution. **Note**: This task has three parts.

 Question d: If we simulate 450 new ignitions in High-hazard areas this season, about how many would we expect to classify as 'fast-growing' (≥10 km in 8 hours)?

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Assume

$$X \sim \text{Lognormal}(\mu = \text{meanlog High}, \sigma = \text{ meanlog High}),$$

with density function $f_{high}(x)$ and CDF $F_{high}(x)$.

The probability that a single new ignition is 'fast-growing' (i.e., spreads 10 km or more in 8 hours) is given by

$$P(X \geq 10) = 1 - F_{high}(10)$$

Now suppose we observe (or simulate) $n = 450$ new ignitions in High-hazard areas this season. Let $X_1, X_2, \ldots, X_{450}$ be random variables, each with the same lognormal distribution as X.

Define indicator variables for 'fast-growing' ignitions:

$$I_i = \begin{cases} 1 & \text{if } X_i \geq 10 \\ 0 & \text{if } X_i < 10 \end{cases} \quad i = 1, \ldots, 450.$$

Then, the total number of 'fast-growing' ignitions, S, is given by

$$S = \sum_{i=1}^{450} I_i$$

Each I_i is a Bernoulli random variable with success probability $P(I_i = 1) = P(X_i \geq 10) = P(X \geq 10)$, so, the expected number of 'fast-growing' ignitions is

$$\mathbb{E}[S] = \sum_{i=1}^{450} \mathbb{E}[I_i] = 450 \cdot P(X \geq 10).$$

Simulate distances for the **High** WHP category using a lognormal distribution. We'll estimate that the upcoming season will have 450 new ignitions and consider a fire fast growing if it spreads 10 km or more in 8 hours.

1. Goal: Generate a large vector of simulated 8-hour distances for the **High** category using rlnorm(). Look up the **High** row's **Mean log** and **SD log** in **Table 1**, and store the result as High_Lognorm. Set the number of ignitions per season to 450.

```
set.seed(2025)
# Test
High_Lognorm <- rlnorm(n = 450, meanlog = 1.907, sdlog = 0.280)
```

```
# Verify
summary(High_Lognorm)
```

 Comment

2. Goal: Use sum() to find the count of new ignitions in the simulation where the wildfire is classified as ‘fast-growing’.

```
# Test
fast_growing_count <- sum(High_Lognorm >= 10)
```

```
# Verify
fast_growing_count
```

 Comment: Based on the simulation, we would expect approximately 30 new ignitions in High-hazard areas to be classified as ‘fast-growing’ (spreading 10 km or more in 8 hours) this season.

3. Goal: Use hist() to create a histogram of High_Lognorm. In one sentence, note how the shape matches your descriptive summaries in Task 1.

The histogram is a discrete approximation to the empirical density of these 450 values. It counts how many simulated distances fall into each of several bins. In other words, it shows what the simulated sample looks like.

```
# Verify
hist(High_Lognorm,
    col   = "blue",
    main   = "Simulated 8-Hour Distances for High WHP (Log
 Normal Model)",
    xlab   = "Distance (km)")
```

 Comment

4. Part C

rlnorm + replicate()

 Question e: In a season with 450 new ignitions **statewide**, what is the probability we see at least 50 fast-growing events (≥10 km in 8 hours) when ignitions can occur in any hazard category according to their statewide area weights?

Let there be L hazard categories indexed by $i = 1, 2, \dots, L$ with:

- statewide area weights p_j such that $\sum_{j=1}^{L} p_j = 1$.

- lognormal spread model in category j:

$X|\text{category} = j \sim \text{Lognormal}(\text{meanlog}_j, \text{sdlog}_j)$.

Consider a season with $N = 450$ new ignitions **statewide**. For each ignition $i = 1, 2, \dots, N$:

1. Draw a hazard category

$$C_i \in \{1, 2, \dots, L\}, P(C_i = j) = p_j.$$

independently across ignitions.

2. Conditional on $C_i = j$, draw an 8-hour straight-line spread distance X_i as

$$X_i|C_i = j \sim \text{Lognormal}(\text{meanlog}_j, \text{sdlog}_j).$$

independently across ignition given their categories.

Define the indicator variable for a 'fast-growing' ignition ($\geq k = 10$ km in 8 hours) as

$$I_i = \begin{cases} 1 & \text{if } X_i \geq k \\ 0 & \text{if } X_i < k \end{cases} \quad i = 1, \dots, 450.$$

Then, the total number of 'fast-growing' ignitions in the season, S, is given by

$$T = \sum_{i=1}^{450} I_i$$

We are interested in the probability that a season has at least 5 fast-growing events:

$$P(T \geq 50).$$

and is approximated by Monte Carlo simulation.

$$\widehat{P(T \geq 50)} \approx \frac{1}{S} \sum_{s=1}^{S} \mathbf{1}\{T^{(s)} \geq 50\},$$

where $T^{(s)}$ is the simulated count of fast-growing ignitions in season s, and S is the total number of simulated seasons.

4.1 Task 1

 Goal: Create a vector of category labels that describe the seven wildfire hazard categories as shown in the Category column of Table 1.

```
# Test
labels <- c("Very Low","Low","Moderate","High","Very High","Extreme","Ultra (Top 1%)")
```

```
# Verify
labels
```

4.2 Task 2

 Goal: Create a vector called prob with from the Probability column from Table 1. The order must match the order of labels.

```
# Test
prob <- c(0.25, 0.25, 0.25, 0.15, 0.05, 0.04, 0.01)  # from Table 1, as proportions
```

```
# Verify
sum(prob)  # should equal 1
```

4.3 Task 3

 Goal: Create vectors of meanlog and sdlog parameters for the lognormal distribution for each category. The order must match the order of labels.

```
# Test
meanlog <- c(0.582, 1.046, 1.451, 1.907, 2.455, 2.866, 3.310)
sdlog <- c(0.472, 0.325, 0.325, 0.280, 0.246, 0.220, 0.212)
```

```
# Verify
meanlog
sdlog
```

4.4 Task 4

 Goal: Set the number of ignitions per season N to 450 and the number of simulations S to 100000. Also, set the fast threshold k to 10 km.

```
# □ Test
N <- 450
S <- 100000
k <- 10
```

4.5 Task 5

□ Goal: Sample categories for one season

Simulate one season's worth of categories, without any distances. Each ignition gets a hazard category chosen at random according to the statewide area weights in prob. Use sample.int() to sample indices from 1:length(labels) and store them in a vector named idx. Then check the first few values and a frequency table to see how often each category appears.

```
# □ Test
idx <- sample.int(length(labels),
                  size    = N,
                  replace = TRUE,
                  prob    = prob)
```

```
# □ Verify
table(labels[idx])
```

□ Comment:

4.6 Task 6

□ Goal: Draw distances for one season

Next, turn those category indices into distances. For each ignition, use rlnorm() with the meanlog and sdlog for that ignition's category. You do this by indexing the parameter vectors with idx. Store the simulated distances in a vector d, then look at the first few values and a summary.

```
# □ Test
d <- rlnorm(N,
            meanlog = meanlog[idx],
            sdlog   = sdlog[idx])
```

```
# □ Verify
head(d)
summary(d)
```

□ Comment:

4.7 Task 7

 Goal: Count fast runs for one season

Now that you have one season of distances in d, count how many ignitions are "fast" (distance at least k = 10 km). Use a logical comparison d >= k to mark fast runs with TRUE/FALSE, then wrap that in sum() to count how many TRUE values there are. Store this in fast_count_one and print it.

```
# Test
fast_flags_one  <- d >= k
fast_count_one  <- sum(fast_flags_one)
```

```
# Verify
fast_count_one
mean(fast_flags_one)
```

4.8 Task 8

Braces { ... } group several lines together so they act as one expression and return the final result.

 Goal: Bundle the three steps above into one block of code that represents a single season. (1) sampling idx, (2) drawing d, and (3) counting fast ignitions. Run it to make sure it still returns a single count.

```
# Test
fast_count_one <- {
  idx <- sample.int(length(labels),
                    size    = N,
                    replace = TRUE,
                    prob    = prob)
  d   <- rlnorm(N,
                meanlog = meanlog[idx],
                sdlog   = sdlog[idx])
  sum(d >= k)
}
```

```
# Verify
fast_count_one
```

 Comment:

4.9 Task 9

 Goal: Use replicate() to simulate many seasons.

Now that the one season computation works and returns a single count, use replicate() to repeat it S times. Store the vector of seasonal counts as counts_fast_mix. Then compute the expected number of fast runs per season and the probability that a season has at least 5 fast-growing events.

```
# Test: Final, complete simulation
set.seed(2025)
counts_fast_mix <- replicate(
  n = S,
  expr = {
    idx <- sample.int(length(labels), size = N, replace = TRUE, prob = prob)
    d   <- rlnorm(N, meanlog = meanlog[idx], sdlog = sdlog[idx])
    sum(d >= k)
  }
)
```

```
# Test: Answer Question
prob_at_least_50 <- mean(counts_fast_mix >= 50)    # Monte Carlo estimate of P(T >= 50)
```

4.10 Task 10

 Goal: Use ggplot2::geom_histogram() to create a histogram of counts_fast_mix. In one sentence, note how the shape matches your descriptive summaries in Task 1.

```
# Test
library(ggplot2)
threshold <- 50  # "at least 5 fast fires"

counts_df <- data.frame(
  fast_count = counts_fast_mix
)

# Pick a reasonable y-position for the annotation (near the top of the tallest bar)
max_y <- max(table(counts_df$fast_count))

ggplot(counts_df, aes(x = fast_count)) +
  geom_histogram(
    binwidth = 1,              # one bin per integer count
    boundary = -0.5,           # centers bins on 0, 1, 2, ...
    color    = "white",
    fill     = "steelblue"
  ) +
  geom_vline(
```

```
  xintercept = threshold - 0.5,  # boundary between 4 and 5
  linetype   = "dashed",
  linewidth  = 0.8
) +
annotate(
  "text",
  x     = threshold + 0.2,
  y     = max_y * 0.9,   # use the precomputed max_y, not ..count..
  label  = paste0("Seasons with \u2265 ", threshold, " fast fires"),
  angle  = 90,
  vjust  = 0.5,
  hjust  = 0
) +
labs(
  title    = "Simulated number of fast-growing fires per season",
  subtitle = paste0(
    "Fast = distance \u2265 10 km in 8 hours; N = 450 ignitions per season\n",
    "Estimated P(season has \u2265 ", threshold, " fast fires) \u2248 ",
    sprintf('%.1f%%', 100 * prob_at_least_50)
  ),
  x       = "Number of fast-growing fires in a season",
  y       = "Number of simulated seasons"
) +
scale_x_continuous(
  breaks = 0:max(counts_df$fast_count)
) +
theme_minimal(base_size = 13) +
theme(
  plot.title      = element_text(face = "bold"),
  panel.grid.minor = element_blank(),
  axis.text.x     = element_text(angle = 90, vjust = 0.5, hjust = 1)
)
```

5. Save and Upload